

Corporación Universitaria Iberoamericana
Programa de Desarrollo de Software

Actividad 4

Estudio de herramientas de calidad de software

Aplicación de análisis estático al caso Therac-25

Presentado por: Nelson Andrés Ramírez Gutiérrez

Curso: Calidad de software

Docente: William Ruiz

Código del curso: 24082026_C1_202634

Bogotá, Colombia
2026

Contenido

1 Introducción

2 Objetivos

3 Metodología y planificación

4 Selección de herramientas

5 Análisis de características y funcionalidades

6 Comparación y selección

7 Aplicación descriptiva de Polyspace

8 Discusión y retroalimentación

9 Consolidación de resultados

10 Conclusiones y recomendaciones

Referencias

1 Introducción

Escogí Therac-25 por la combinación entre un equipo médico y un error de software. Me impactó que varios fallos graves de ingeniería terminaran en sobredosis masivas de radiación. El caso no se queda únicamente en el código: involucra decisiones médicas, funcionamiento del equipo y seguridad de los pacientes. Al volver al estudio de Leveson y Turner (1993), entendí que los mensajes al operador y la respuesta a los primeros incidentes también hicieron parte del problema.

Yo ya había trabajado con SonarQube en la empresa, tanto en Python como en .NET. Esa experiencia me sirvió como punto de comparación. Polyspace no la conocía y me llamó la atención porque se concentra en C y C++ embebido. Por esa diferencia decidí estudiarla para el módulo del Therac-25 (MathWorks, s. f.-c).

El ejercicio tiene dos límites que deben quedar claros. No tuve acceso al código original del Therac-25 ni ejecuté una licencia de Polyspace. Por eso elaboré un módulo simulado y definí qué resultados esperaba observar durante un piloto. Las alertas incluidas más adelante son hipótesis de revisión, no resultados reales atribuidos a la herramienta.

2 Objetivos

2.1 Objetivo general

Analizar y seleccionar una herramienta de calidad de software pertinente para el caso Therac-25 y describir su aplicación en la identificación y tratamiento de defectos de un módulo de control simulado.

2.2 Objetivos específicos

- Comparar cuatro herramientas que puedan revisar código C o C++ y reconocer para qué sirve cada una.
- Explicar mediante la matriz por qué seleccioné una opción para el caso Therac-25.

- Representar tres riesgos en un módulo simulado y definir qué evidencia pediría para cerrar una alerta.

3 Metodología y planificación

No usé todas las fuentes para lo mismo. Del artículo de Leveson y Turner (1993) tomé los riesgos del caso. Los libros de la unidad me ayudaron a ordenar el análisis de calidad (Piattini-Velthuis & García-Rubio, 2015; Pressman & Maxim, 2021). Para conocer las funciones de los productos fui a sus páginas oficiales. Después de esa lectura definí los criterios de comparación.

Trabajé por etapas y dejé cada resultado como entrada de la etapa siguiente. Primero retomé los riesgos de la actividad 2 y después preparé una ficha por herramienta. La matriz se utilizó como una orientación para que el lector pudiera ver la comparación en números. Para cada criterio asigné una calificación entre 1 y 5, la multipliqué por su peso y sumé los resultados. Por ejemplo, una calificación de 5 en un criterio de 30 % aporta 1,50 al total. Los puntajes reflejan mi valoración del caso; no corresponden a una regla certificada ni a una prueba comercial.

ISO/IEC 25010:2023 me sirvió para nombrar aspectos como fiabilidad, seguridad operacional y mantenibilidad (International Organization for Standardization [ISO], 2023). También tuve en cuenta la calidad de los datos explicada por Salcedo Cifuentes et al. (2018). En este caso el dato importante es el estado interno del equipo: si queda desactualizado, el programa puede decidir con una condición que ya cambió.

Tabla 1*Comparación retrospectiva de casos potenciales*

Caso	Relación con calidad	Disponibilidad	Aplicación de herramientas	Decisión
Therac-25	Muy alta: software médico y seguridad	Alta	Alta en simulaciones C y análisis estático	Seleccionado
Ariane 5 vuelo 501	Alta: errores del software de orientación	Alta	Alta para desbordamientos y pruebas	Alternativa
Mars Climate Orbiter	Alta: interfaces y unidades	Alta	Media para análisis estático aislado	Alternativa
Equifax 2017	Alta: vulnerabilidad y parches	Alta	Alta para SAST y dependencias	Alternativa

Nota. Comparación retrospectiva basada en Leveson y Turner (1993), European Space Agency (1996), National Aeronautics and Space Administration (2025) y Federal Trade Commission (2019). Se mantiene Therac-25 por su relevancia para la seguridad humana, utilidad e información disponible. El alcance descriptivo delimita la viabilidad y evita exponer pacientes. Las valoraciones de la tabla son propias.

Tabla 2*Planificación de la actividad*

Etapas	Momento	Actividad	Recursos	Responsable
1	Día 1	Volver a la actividad 2 y anotar los riesgos que usará	Informe anterior y artículo del caso	Nelson A. Ramírez
2	Día 2	Preparar una ficha breve de cada herramienta	Sitios oficiales y textos de la unidad	Nelson A. Ramírez
3	Día 3	Definir pesos, calificar y comprobar las sumas	Matriz de comparación	Nelson A. Ramírez
4	Día 4	Representar tres riesgos en el módulo simulado	Esquema en C y documentación de Polyspace	Nelson A. Ramírez
5	Día 5	Revisar citas, conclusiones y presentación	Lista de referencias y lista de control	Nelson A. Ramírez

Nota. Plan de ejecución individual; los días representan la secuencia prevista.

4 Selección de herramientas

Reduje la lista a productos capaces de revisar C o C++. Luego miré si seguían el flujo de datos y si podían advertir problemas entre tareas. Me interesaban esos puntos porque una prueba normal no siempre repite el momento exacto en que aparece una condición concurrente. De todas formas, el análisis estático es solo una parte de la verificación (Pressman & Maxim, 2021).

Tabla 3

Herramientas preseleccionadas

Herramienta	Tipo y alcance	Problemas que ayuda a resolver	Integración principal
Cppcheck	Análisis estático C/C++ de código abierto	Comportamiento indefinido y construcciones peligrosas; prioriza pocos falsos positivos	CLI, compilación y reportes XML/HTML
SonarQube Server	Plataforma de calidad continua con analizador CFamily	Bugs, vulnerabilidades, mantenibilidad, complejidad y centralización de métricas	CI/CD, servidor y perfiles de calidad
Coverity	SAST comercial para múltiples lenguajes	Defectos de flujo, seguridad y reglas de codificación	Compilación, CI y revisión centralizada
Polyspace	Análisis y verificación de C/C++ embebido	Errores de ejecución, concurrencia, reglas MISRA y demostración de ausencia de ciertas fallas	IDE, escritorio, servidor y CI/CD

Nota. Síntesis basada en Cppcheck (s. f.-a), SonarSource (s. f.-a), Black Duck (s. f.-b) y MathWorks (s. f.-a, s. f.-c).

5 Análisis de características y funcionalidades

5.1 Cppcheck

Comencé con Cppcheck porque es de código abierto y puede incorporarse con facilidad a un ejercicio académico. El proyecto indica que revisa comportamiento indefinido y construcciones peligrosas en C y C++, y que puede producir salidas XML e informes HTML (Cppcheck, s. f.-a). Lo usaría como primera revisión por su costo y sencillez. No lo tomaría como evidencia única en un dispositivo médico, pues su propio alcance no cubre todo el campo del análisis estático. Su edición abierta es gratuita, funciona en Windows y entornos POSIX y dispone de manual y recursos públicos del proyecto (Cppcheck, s. f.-b).

5.2 SonarQube Server

SonarQube no era una herramienta desconocida para mí. La he utilizado en la empresa con código Python y .NET, donde resulta útil para concentrar reglas, métricas y problemas de calidad. Para esta actividad revisé además su analizador CFamily, que trabaja con C, C++ y Objective-C y puede recibir cobertura de herramientas externas (SonarSource, s. f.-a). Aunque conozco su utilidad empresarial, en el caso del Therac-25 buscaba una opción más enfocada en software embebido crítico. CFamily requiere Developer o superior; el analizador funciona en Windows, Linux y macOS y cuenta con Sonar Community (SonarSource, s. f.-b, s. f.-c).

5.3 Coverity

Coverity también es comercial, pero está orientado al análisis a escala y a la administración centralizada de hallazgos en lenguajes como C y C++. Su documentación contempla revisiones frente a estándares de codificación (Black Duck, s. f.-b). En una organización grande esta trazabilidad puede ser valiosa; para el ejercicio individual, el licenciamiento, la infraestructura y la configuración representan una barrera mayor que en Cppcheck. Coverity Analysis admite Windows, Linux y macOS en versiones compatibles; la licencia y el soporte se gestionan con Black Duck Community (Black Duck, s. f.-a, s. f.-c).

5.4 Polyspace

Al leer la documentación de Polyspace encontré dos funciones que encajaban con el ejercicio. Bug Finder busca fallas de ejecución y concurrencia. Code Prover se usa para comprobar propiedades en código crítico (MathWorks, s. f.-a, s. f.-c). Yo no ejecuté el producto, pero esas funciones coincidían mejor con el módulo simulado. Por eso recibió el puntaje más alto. Polyspace Desktop admite Windows, Linux y macOS, mediante compra o licencia de prueba; ofrece documentación, MATLAB Answers y soporte (MathWorks, s. f.-b, s. f.-c).

6 Comparación y selección

Quise que la tabla mostrara por qué una herramienta quedaba encima de otra. Puse el riesgo en primer lugar con 30 %. La integración recibió 20 %. Compatibilidad y facilidad de uso quedaron con 15 % cada una; costo y soporte, con 10 % cada uno. Esta distribución es mi criterio para el caso, no una fórmula tomada de una norma.

Tabla 4

Matriz de decisión ponderada de 1 a 5

Criterio	Peso	Cppcheck	SonarQube	Coverity	Polyspace
Cobertura de riesgos críticos	30 %	3	3	4	5
Integración y trazabilidad	20 %	3	5	5	5
Compatibilidad con C y C++	15 %	4	4	4	5
Facilidad de uso	15 %	4	4	3	3
Costo y acceso	10 %	5	3	2	2
Soporte especializado	10 %	3	4	5	5
Total ponderado	100 %	3,50	3,80	3,95	4,40

Nota. Valoración orientativa elaborada para este caso. Total = suma de cada calificación (1 a 5) multiplicada por su peso.

Al hacer la suma, Polyspace obtuvo 4,40; Coverity, 3,95; SonarQube, 3,80; y Cppcheck, 3,50. El primer resultado coincide con los riesgos que prioricé, en especial concurrencia y errores de ejecución (MathWorks, s. f.-a, s. f.-c). El resultado no sirve igual

para cualquier proyecto. Con poco presupuesto empezaría por Cppcheck; si el problema fuera reunir métricas de un equipo, evaluaría primero SonarQube.

7 Aplicación descriptiva de Polyspace

7.1 Alcance y supuestos

Para hacer concreta la aplicación preparé un esquema conceptual en C. No intenté reconstruir el programa propietario. Representé tres situaciones tomadas del análisis histórico: dos tareas que comparten datos, un contador capaz de volver a cero y una validación que separa la comprobación del modo y la configuración física. Leveson y Turner (1993) describen estos tipos de problemas en su investigación del Therac-25.

7.2 Fragmento conceptual del módulo

Tabla 5

Elementos del módulo simulado y peligros asociados

Elemento simulado	Comportamiento vulnerable	Peligro
uint8_t contador_pruebas	Incrementa y vuelve a cero después del límite	Una comprobación periódica puede omitirse en la ventana de reciclado.
estado_modos global	Dos tareas leen y escriben sin exclusión	La tarea de tratamiento puede usar una combinación de estados de momentos distintos.
habilitar_haz()	Valida condiciones en instrucciones separadas	Un cambio entre comprobación y uso invalida el supuesto de seguridad.
codigo_error numérico	Permite continuar sin estado seguro	La interfaz no comunica la gravedad ni impide un reintento peligroso.

Nota. Relación con ISO (2023): contador y habilitación, seguridad operacional; datos compartidos, fiabilidad; mensajes de error, capacidad de interacción. H-05 aborda mantenibilidad. El modelo en C es pedagógico; el original utilizaba ensamblador PDP-11 (Leveson & Turner, 1993).

7.3 Configuración propuesta

- Primero reproduciría el compilador, los tipos de datos, las macros y las entradas del módulo. Sin esa configuración el informe podría revisar un programa diferente al real.

- Después ejecutaría Bug Finder para revisar memoria, valores numéricos y concurrencia. En las funciones que habilitan la salida usaría Code Prover y las reglas MISRA C pertinentes (MathWorks, s. f.-a, s. f.-c).
- Por último asignaría las alertas, registraría cada decisión y repetiría el análisis después de corregir el código. Una alerta crítica abierta impediría cerrar el ejercicio.

Figura 1

Flujo de aplicación y cierre de hallazgos



Nota. Elaboración propia a partir del flujo de análisis descrito por MathWorks (s. f.-a).

7.4 Resultados esperados del piloto

La tabla siguiente registra lo que buscaría durante el piloto. Debido a que no ejecuté Polyspace ni dispuse del código original, no presento estas filas como alertas obtenidas. Para cada riesgo definí una corrección posible y la evidencia que exigiría antes de cerrarlo. Esta separación permite distinguir mi propuesta de los resultados que solo podrían obtenerse al analizar un proyecto real.

Tabla 6*Hallazgos esperados, correcciones y evidencia de cierre*

ID	Hallazgo esperado	Severidad	Corrección propuesta	Evidencia de cierre
H-01	El contador puede volver a cero y omitir un control	Crítica	Dejar de usarlo como control de seguridad y representar el estado de forma explícita	Revisión del requisito de seguridad y prueba de 255 a 0
H-02	Dos tareas modifican estado_modos	Crítica	Sincronizar la lectura y la actualización del estado compartido	Revisión de concurrencia y prueba de cambios rápidos
H-03	El estado cambia después de validarlo	Crítica	Unir la validación con la habilitación y mantener un enclavamiento independiente	Propiedad comprobada y prueba integral
H-04	El código de error permite reintentar	Alta	Detener la operación, registrar el contexto y mostrar una acción comprensible	Prueba de interfaz y recuperación segura
H-05	La función reúne demasiadas decisiones	Media	Separar lectura, validación, decisión y actuación	Revisión por pares y menor complejidad

Nota. Volver a cero es válido para un entero sin signo en C; el peligro es omitir la comprobación. Su verificación requiere un requisito o aserción de seguridad explícita.

7.5 Criterios para liberar el módulo

- No liberaría el módulo con alertas críticas o altas abiertas. Un falso positivo necesitaría explicación y revisión de otra persona.
- Para cerrar un hallazgo conservaría el riesgo, la corrección, el nuevo análisis y una prueba que incluya límites, cambios rápidos y reinicio.
- Aunque todas las verificaciones pasaran, el resultado solo correspondería al módulo simulado y no autorizaría el uso clínico del equipo.

8 Discusión y retroalimentación

Como hice el trabajo solo, tuve que revisar mis propios puntajes. Lo más difícil fue entender cómo se unieron los cálculos del programa, las decisiones de los ingenieros y la falta de pruebas antes del uso con pacientes. Para revisar el resultado me pregunté si cada alerta representaba un peligro real, qué evidencia demostraría la corrección y si existía una defensa cuando el software fallara. En un proyecto real esa revisión tendría que hacerla también otra persona.

9 Consolidación de resultados

La matriz me ayudó a separar usos que al principio veía mezclados. SonarQube me resulta conocido para el seguimiento diario de la calidad; Cppcheck puede apoyar una revisión rápida; Coverity está pensado para un análisis comercial más amplio; y Polyspace se acerca al tipo de control crítico estudiado. Elegir Polyspace para este ejercicio no significa que las demás herramientas dejen de ser útiles.

Tabla 7

Síntesis de aprendizajes y recomendaciones

Aprendizaje	Aplicación al Therac-25	Recomendación
El analizador cubre solo una parte del problema	El caso combinó software, equipo, interfaz y decisiones de gestión	Usar sus alertas junto con pruebas y análisis de peligros
Una mala configuración cambia el informe	Los tipos de datos y las tareas influyen en los errores buscados	Registrar compilador, macros, entradas y supuestos
Cerrar una alerta requiere evidencia	Los primeros incidentes no se relacionaron oportunamente	Guardar responsable, corrección, reanálisis y prueba
Los resultados críticos necesitan otra revisión	Una clasificación equivocada puede ocultar un peligro	Pedir aprobación independiente antes del cierre

10 Conclusiones y recomendaciones

La suma dio 4,40 para Polyspace. Su ventaja en esta comparación fue el trabajo con C y C++ embebido y la revisión de concurrencia (MathWorks, s. f.-a, s. f.-c). SonarQube sigue siendo valioso según mi experiencia en la empresa, pero responde a una necesidad diferente. Para una compra real probaría las dos herramientas con código del proyecto antes de decidir.

Con el módulo pude formular tres preguntas: qué pasa si el contador regresa a cero, quién cambia el estado compartido y qué se comprueba justo antes de habilitar la salida. Esas preguntas muestran por qué no conviene dejar las pruebas para el final. El accidente incluyó software, ingeniería y uso clínico, no una sola falla aislada (Leveson & Turner, 1993).

Mi aprendizaje fue que la calidad acompaña toda la vida del software. En un equipo médico aplicaría fallos seguros, estándares actuales, validación de errores y pruebas de punta a punta. Antes de trabajar con pacientes exigiría además una auditoría externa o una revisión realizada por personal autorizado. El objetivo sería reducir tanto el error técnico como el humano.

Referencias

- Black Duck. (s. f.-a). *Installing Coverity Analysis*. Coverity Documentation. <https://docs.blackduck.com/r/coverity/latest/coverity-documentation/installing-coverity-analysis.html>
- Black Duck. (s. f.-b). *Running coding-standard analyses*. Coverity Documentation. <https://docs.blackduck.com/r/coverity/latest/coverity-documentation/running-coding-standard-analyses.html>
- Black Duck. (s. f.-c). *Supported platforms for Coverity Analysis*. Coverity Documentation. <https://docs.blackduck.com/r/coverity/2026.3/coverity-documentation/supported-platforms-for-coverity-analysis.html>
- Cppcheck. (s. f.-a). *Cppcheck: A tool for static C/C++ code analysis*. <https://cppcheck.sourceforge.io/>
- Cppcheck. (s. f.-b). *Cppcheck manual*. <https://cppcheck.sourceforge.io/manual.html>
- European Space Agency. (1996). *In brief: Ariane-501: The Inquiry Board's report*. <https://www.esa.int/esapub/bulletin/bullet87/inbrie87.htm>
- Federal Trade Commission. (2019, 22 de julio). *Equifax to pay \$575 million as part of settlement with FTC, CFPB, and states related to 2017 data breach*. <https://www.ftc.gov/news-events/news/press-releases/2019/07/equifax-pay-575-million-part-settlement-ftc-cfpb-states-related-2017-data-breach>
- International Organization for Standardization. (2023). *Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model* (ISO/IEC 25010:2023). <https://www.iso.org/standard/78176.html>
- Leveson, N. G., & Turner, C. S. (1993). An investigation of the Therac-25 accidents. *Computer*, 26(7), 18-41. <https://doi.org/10.1109/MC.1993.274940>
- MathWorks. (s. f.-a). *Code verification*. <https://www.mathworks.com/help/overview/code-verification.html>

MathWorks. (s. f.-b). *Install Polyspace Desktop Products*. <https://www.mathworks.com/help/bugfinder/install/install-products-required-for-polyspace-analysis-on-desktop.html>

MathWorks. (s. f.-c). *Polyspace*. <https://www.mathworks.com/products/polyspace.html>

National Aeronautics and Space Administration. (2025, 15 de septiembre). *Mars Climate Orbiter*. <https://science.nasa.gov/mission/mars-climate-orbiter/>

Piattini-Velthuis, M., & García-Rubio, F. (2015). *Calidad de sistemas de información*. RA-MA Editorial.

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería de software: Un enfoque práctico*. McGraw-Hill Interamericana.

Salcedo Cifuentes, M., Gutiérrez Martínez, M. I., & Cardona Orozco, S. L. (2018). *La calidad del dato en los sistemas de información de convivencia y seguridad ciudadana*. Programa Editorial Universidad del Valle. <https://doi.org/10.25100/peu.239>

SonarSource. (s. f.-a). *C/C++/Objective-C analysis overview*. SonarQube Server Documentation. <https://docs.sonarsource.com/sonarqube-server/analyzing-source-code/languages/c-family/overview>

SonarSource. (s. f.-b). *Download & install SonarQube*. <https://www.sonarsource.com/products/sonarqube/downloads/>

SonarSource. (s. f.-c). *Prerequisites*. SonarQube Server Documentation. <https://docs.sonarsource.com/sonarqube-server/analyzing-source-code/languages/c-family/prerequisites>